

**Faculty of Engineering
ECCE Department**

**EEN 527
Embedded System Design
Section A**

Instructor:

Dr. Abdallah Kassem

Literature Review

**FPGA Based Intrusion Detection and
Prevention System for CAN Bus Networks**

Submitted by:

Anthony El Chakar #20231589

Charbel Estephan #20231222

Joe Geagea #20201000

1 Introduction

This review covers existing research on intrusion detection and prevention systems for the Controller Area Network (CAN) protocol, with a focus on hardware based and FPGA based implementations. CAN networks power most embedded systems in vehicles and industrial equipment, but the protocol has no built in authentication or encryption. This gap leaves it open to spoofing, denial of service, and replay attacks.

The review draws on IEEE Xplore, ScienceDirect, and arXiv, and it covers work published between 2012 and 2026. It includes conference papers, journal articles, and preprints on CAN intrusion detection and prevention, whether the design runs in software, on a microcontroller, on an FPGA, or on an application specific circuit. It excludes intrusion detection work aimed at other automotive networks such as FlexRay or Automotive Ethernet, and it treats message authentication and encryption schemes only in passing, since they solve the same problem through a different mechanism than the detection and prevention systems this review examines.

The rest of this document follows a fixed structure. Section 2 lays out the background a reader needs on the CAN protocol itself, the categories of attack the literature studies, and the built in mechanism every prevention design in this review relies on to stop a bad frame once it finds one. Section 3 surveys the related work in three groups, rule based and signature based detectors, machine learning based detectors, and hardware implementations of either kind, then closes with the gap this project targets. Section 4 succinctly restates this limitation and details how the proposed hardware architecture addresses it.

2 Background

CAN organizes traffic as broadcast frames rather than addressed packets. Each frame carries an arbitration field of 11 or 29 bits that both identifies the message and sets its priority, since the frame with the lowest identifier wins the bus when several nodes try to transmit at the same time. A control field states the data length, up to eight bytes of data

follow, and a cyclic redundancy check, an acknowledge field, and an end of frame field close the frame out. Every node on the bus receives every frame, and the protocol has no field for a sender address, a signature, or an encryption key. Robert Bosch GmbH, who introduced the protocol, built it for a closed vehicle environment where every node was assumed trustworthy. That assumption no longer holds once a node can be reached through the On Board Diagnostics port, an infotainment system, or a cellular modem.

Attacks on CAN fall into a small number of recurring categories. Ileri et al. group them as denial of service, fuzzy, malfunction, replay, and masquerade attacks [1]. A denial of service attack floods the bus with a high priority identifier so that legitimate frames lose arbitration. A fuzzy attack injects frames with random identifiers and random payloads to trigger unexpected behavior. A malfunction or spoofing attack reuses a legitimate identifier with a forged payload, so a receiving node accepts data it should not. A replay attack retransmits a previously captured frame. A masquerade attack silences the legitimate sender of an identifier and takes its place. Miller and Valasek showed that an attacker can reach the bus without any physical access at all, disabling brakes, altering the instrument cluster, and taking other unsafe actions through the cellular, Bluetooth, and Wi Fi interfaces built into modern telematics units [2]. Their work moved the field's threat model away from an attacker who needs a wrench and a laptop under the dashboard and toward one who needs only an internet connection, and it is one of the main reasons intrusion detection research on CAN accelerated afterward rather than treating physical bus access as an unrealistic assumption. Seo et al. built one of the datasets most later work relies on by recording normal traffic from a Hyundai YF Sonata and injecting denial of service, fuzzy, and RPM or gear spoofing frames at fixed intervals [3]. Because that same vehicle and that same set of attack types recur across so many of the papers surveyed below, this review treats it as the default benchmark and calls out any paper that departs from it. Donadel et al. separate attacks by the access level they need, distinguishing frame injection, which any node with bus access can attempt, from single bit attacks, which require an attacker who can drive individual

bits on the physical lines [4]. Araujo Filho et al. state the threat model that most of the papers in this review adopt. Their attacker can sniff the bus, learn the identifiers and timing patterns in use, inject new frames, and modify a frame in transit while updating its checksum [5]. This project adopts the same threat model, since it maps directly onto a device that taps the physical transmit and receive lines, reads every bit as it passes, and decides whether to act before the frame ends.

The literature groups defenses into two broad classes. Signature based systems compare traffic against a set of known attack patterns and flag a match. They detect known attacks reliably but miss anything outside their signature set. Anomaly based systems learn what normal traffic looks like and flag deviations from it. This second class covers most of the work reviewed here, and it splits further into systems that model message timing and frequency, systems that check a message against the physical or electrical fingerprint of the sending node, and systems that extract statistical or learned features from the identifier, timing, and payload fields and classify them with a machine learning model. A random forest, the model class this project builds around, sits in that last group. It reads a set of features computed from a frame, sends the features down many decision trees trained on different random subsets of the training data, and takes a majority vote across the trees, which is what gives it a reputation for resisting the overfitting that a single deep decision tree suffers from.

CAN also gives every defense a built in tool for stopping an attack once it is caught. The bit stuffing rule requires a change of level after five consecutive identical bits, so a node can force an error condition by sending six dominant bits in a row. Every other node reads this as a protocol violation, discards the frame in progress, and sends its own error frame, and repeated violations from one node raise its error counters until it enters the bus off state and stops transmitting. This mechanism matters for a prevention system for two reasons beyond simply blocking one bad frame. First, it needs no cooperation from the rest of the bus, since every CAN controller already implements bit stuffing and error counting as part of the protocol, so a single added device can trigger it without touching any other node's firmware.

Second, it is available the instant the offending bits are seen, which is why the timing budget in Section 3.3 measures against the frame's own end rather than against some later software response. Section 3.3 returns to this mechanism, since most of the prevention designs in this review use it.

Two adjacent lines of work fall outside detection and prevention but shape why they matter. Message authentication schemes add a code to each frame so a receiver can reject anything it cannot verify, at the cost of extra bits that CAN's small payload struggles to spare. Kurachi et al. built one such scheme, Ca-CAN, around a centralized node that authenticates messages on behalf of the rest of the bus [14]. Physical layer schemes such as bitrate hopping change the bus speed on a shared secret sequence so an unsynchronized attacker cannot follow the traffic at all [15]. Both approaches need every node on the bus to cooperate, which is a heavier requirement than adding one detection device, and this is part of why detection and prevention systems that tap the bus from a single point remain an active research direction.

3 Related Work

3.1 Rule Based and Signature Based Detection

The earliest CAN specific detectors work from a small number of hand chosen features rather than a trained model. Song et al. showed that the interval between successive messages with the same identifier stays stable under normal operation and that fuzzy and denial of service attacks disturb it enough to raise no false alarms in their tests [6]. This result matters beyond its own numbers, since it shows that a feature as simple as a timestamp difference, something a small microcontroller can compute with a single subtraction, already carries enough signal to separate two of the four attack categories from normal traffic without any learned model at all. Lee et al. built on this idea by measuring how far each frame's interval and payload Hamming distance deviate from the averages recorded for its identifier in normal traffic, then passing those two deviations to a three tree random forest trained in WEKA to flag spoofing [7]. Because both features come from

comparing a new frame against a running average kept per identifier, the detector needs only a small table indexed by identifier and a handful of arithmetic operations per frame, which is part of why Section 3.3 shows this same design translating cleanly into dedicated hardware. Section 3.3 covers the hardware built around this same feature set.

Detection based on message ownership offers a different rule. Abbott McCune and Shay gave every node a whitelist of the identifiers it owns and flagged any frame carrying one of those identifiers while the owning node was not transmitting, catching a replay directly without needing a learned model [8]. Pese et al. built MichiCAN on the same principle at the bit level, monitoring the arbitration field bit by bit from inside the CAN controller itself, which lets it flag a denial of service attack before the identifier field finishes rather than waiting for the whole frame [9]. Donadel et al. take the most direct rule of all with CANTXSec, an officer that watches which electronic control unit drives the transmit line for each identifier and flags any frame whose sender does not match the expected node, without any learning step at all [4].

These designs share a strength and a weakness. Because they check a fixed rule rather than a learned pattern, their behavior is easy to reason about and their detection is immediate once the relevant bits arrive. Because the rule is fixed, they only catch what their designer anticipated, and Donadel et al. note that a purely rule based officer needs no training data at all, in contrast with every machine learning design discussed next [4]. This tradeoff is why rule based detection alone does not fit a project built around a random forest. A whitelist or an ownership rule catches masquerade and replay attacks cleanly but has nothing to say about a fuzzy attack that forges a payload for an identifier the correct node is still transmitting, which is exactly the gap a learned classifier is meant to close.

3.2 Machine Learning Based Detection

Most recent CAN intrusion detectors apply a classifier to features drawn from the identifier, the timing, or the payload. Seo et al. converted CAN identifiers into a simple image and trained GIDS, a pair of generative adversarial discriminators, where the first discriminator learns known attacks and the second

learns only from normal traffic so it can flag attacks it never saw [3]. Their design shows that even a shallow, hand engineered transformation of the identifier field carries enough structure for a neural network to work with, though the two discriminator training loop that GIDS needs adds cost that a simpler tree based classifier avoids.

Some of the reviewed designs train only on normal traffic, which lets them flag attacks without ever seeing a labeled example, while others train on labeled attacks. Araujo Filho et al. compared a one class support vector machine against an isolation forest and found the isolation forest reached accuracy above 99 percent and an F1 score above 97 percent while reading only the first five payload bytes of each frame, which matters because fewer bytes read means less time spent before a decision can be made [5]. Ileri et al. built MetaCAN, an XGBoost classifier tuned with a hybrid of particle swarm optimization and cuckoo search, and reported accuracies from 0.984 to 0.99999 across three public datasets, though training took between roughly three and ninety four hours depending on the dataset [1]. XGBoost and random forest both build an ensemble of decision trees, but XGBoost grows each new tree to correct the errors of the ones before it, while a random forest trains every tree independently on a random subset of the data and simply averages their votes. The dependence between XGBoost trees exists only during training. Once trained, both models can evaluate every tree in parallel, since a random forest combines its trees by a vote and XGBoost combines them by a sum. The difference that matters for hardware is in that last step. Each random forest leaf holds a class label, so combining the trees needs only a small vote counter, while each XGBoost leaf holds a real valued score, so combining them needs an adder tree and a final comparison.

These results rest on a small set of shared benchmarks. The Car Hacking Dataset, recorded from the same Hyundai Sonata used by Seo et al., remains the default benchmark for denial of service, fuzzy, and spoofing attacks [3]. Because nearly every paper in this section reports accuracy on one or more of these same datasets, the differences in reported accuracy trace mainly to whether a model was trained on the specific attack it was later tested against, not to which algorithm was used, a pattern that carries di-

rectly into the hardware implementations discussed next.

3.3 Hardware Implementations

A growing body of work moves these classifiers off a general purpose processor and onto dedicated hardware, chiefly to meet the timing budget that a CAN frame allows. Araujo Filho et al. derive that budget directly from the protocol. If a detector reads the first N of eight payload bytes, the time left to decide and to send an error frame before the current frame's end of frame field arrives equals 19 plus 8 times the quantity 8 minus N , measured in bit times, minus whatever delay the hardware itself adds. At 500 kilobits per second and five payload bytes this comes to about 86 microseconds, and their isolation forest running on a Raspberry Pi 4 averaged 78 to 79 microseconds across the three attacks they tested, meeting the budget with margin to spare [5]. That margin came from software running on a general purpose processor with an operating system underneath it, and it sets the baseline every hardware design discussed below is trying to beat by a wider margin, at a lower power draw, or both.

Khandelwal and Shreejith built an FPGA based intrusion detection system on a hybrid device that combines an ARM processor with programmable logic acting as a single electronic control unit. Their design, a quantized convolutional network reading a window of four identifiers, reaches an F1 score of 0.9996 for denial of service and 0.9922 for fuzzing at 0.43 milliseconds per frame while drawing 2.0 watts, against 54 watts for the same model on a GPU [10]. The quantization step, shrinking every weight down to a few bits instead of the 32 bit floating point values a GPU trains with, is what lets a neural network of this size fit onto programmable logic at all, and it is the main reason the power gap against the GPU is as large as it is.

Other groups report similar gains. Rangsikunpum et al. paired a binarized network with a generative adversarial discriminator on a low cost Zed-board, reaching 0.169 milliseconds per decision at 2.09 watts using no dedicated multiplier blocks at all [11]. Avoiding dedicated multiplier blocks matters for a board like the DE10 Lite, since a design that leans on ordinary programmable logic instead

of a fixed number of hardware multipliers has more headroom to grow without running into a hard resource ceiling.

Tree based designs follow a separate and generally lighter path. Lee et al. synthesized a random forest trained with WEKA directly into Verilog hardware and reached 98.2 percent accuracy in simulation with three trees [7]. Because a decision tree is just a sequence of comparisons against stored thresholds, each node maps onto a small comparator circuit rather than the multiply and accumulate operations a neural network needs, which keeps resource use low even before any quantization step is applied. A later paper by largely the same group integrates a five tree, depth nine random forest directly into a CAN controller and fabricates the result as a 180 nanometer application specific circuit, averaging 1.86 microseconds per detection using 4437 lookup tables against 56733 for a comparable neural network design on the same device [12]. That thirteen fold difference in lookup table count, for two designs solving the same classification problem on the same fabrication process, is the clearest evidence in this review that model choice, not just platform choice, drives resource cost. Jang et al. built a lightweight convolutional network sized for a single CAN frame, cutting parameter count from 4045 to 1789 against their own earlier design, and report a decision 5.68 microseconds after the last payload byte arrives, fabricated in a 28 nanometer process at 34.68 nanojoules per inference [13].

Abbott McCune and Shay had already split a CAN defense across the same two kinds of device this project compares. Their prototype runs the ownership rule from Section 3.1 as VHDL on a small Altera FPGA board that follows the receive and transmit lines bit by bit, pairs it with an Arduino Uno microcontroller that makes decisions for the node, and lets the node force a spoofed frame to error out [8]. They verify the design in ModelSim and do not report latency or power, and because the rule is fixed rather than learned, the design gives no information about how a trained classifier behaves on the same split.

Table 1 collects the platform, model, and reported cost for the hardware designs above. The latency figures are not directly comparable, since the studies measure from different starting points, some

from the arrival of the first payload byte, others from the last, and one from the end of the data field, and a reader comparing raw numbers across rows should keep that difference in mind rather than reading a smaller number as simply faster hardware.

Several of the hardware designs above also stop the traffic they flag rather than only reporting it, using the same error frame mechanism described in Section 2. Araujo Filho et al. force six dominant bits from a Raspberry Pi through an MCP2551 transceiver [5]. Lee et al. and Jang et al. build the same response into their controller integrated circuits, and Lee et al. show the resulting error flag on an oscilloscope [12, 13]. Because repeated blocking does not stop a node from retrying, several of these designs go further and drive the offending node into the bus off state through repeated error frames rather than treating one blocked frame as a complete response [4, 9, 12]. This distinction between detection and prevention matters for this project’s own scope. A design that only reports an attack still leaves the malicious frame on the bus for every other node to act on, while a design that forces an error frame in time removes it before any node can, which is why every build in this project targets prevention rather than detection alone.

3.4 Gaps in the Current Literature

None of the papers reviewed above compares an FPGA only implementation of the same detection model against a microcontroller only implementation and a combined FPGA plus microcontroller implementation. The FPGA based designs in Section 3.3 report their own latency and power figures in isolation, most often against a GPU or against each other, and the two application specific circuit designs integrate their classifier behind an existing CAN controller rather than testing whether a plain microcontroller could run the same model within the same timing budget [10, 12, 13]. The rule based, bit level designs that tap the transmit and receive lines directly, CANTXSec and MichiCAN, run on a microcontroller rather than an FPGA, and neither paper tests the same detection logic on both platforms to see where the timing budget breaks down [4, 9]. Abbott McCune and Shay pair an FPGA with a microcontroller, but they use the pair as a single design

with a fixed rule, and they measure neither device on its own [8]. This leaves open a direct, controlled answer to a practical design question. Given the same random forest model and the same bit level tap into the physical CAN lines, does an FPGA, a microcontroller, or a combination of the two give the best balance of detection latency, resource cost, and power draw. Answering that question needs one model held constant across all three builds, since comparing an FPGA running one model against a microcontroller running a different one would leave open whether any measured difference came from the platform or from the model, which is exactly the confound every paper surveyed above leaves unresolved.

4 Summary and Positioning of This Work

The literature agrees on two points more than any other. Systems trained on normal traffic alone, whether an isolation forest, an autoencoder, or a rule based tap on the transmit line, generalize to attacks they never saw, while systems trained with labeled attacks reach higher accuracy only on the attacks they trained on. Once a classifier is committed to an application specific circuit, detection lands in the single digit microsecond range, well inside the timing budget the protocol allows. The FPGA designs in Table 1 took 0.169 and 0.43 milliseconds per decision, both above the 86 microsecond budget from Section 3.3, but both ran neural networks. So the open question is whether a lighter, tree based model can bring an FPGA or a microcontroller inside that budget, and which platform then delivers that speed at the lowest cost and power for a given accuracy target. The random forest results in Section 3.3 point toward an answer, since a tree based model reached microsecond level detection with an order of magnitude fewer lookup tables than a neural network solving the same problem, which suggests the model class matters as much to that answer as the platform does.

This project addresses the gap identified above by building a random forest classifier, the same model class that two prior hardware papers have already committed to silicon, and evaluating it on three

Table 1: Reported platform, model, latency, and power or energy across the hardware designs reviewed above.

Study	Platform	Model	Latency	Power or Energy
Araujo Filho et al. [5]	Raspberry Pi 4	Isolation forest	78 to 79 microseconds	Not reported
Khandelwal and Shreejith [10]	Ultra96 V2	Quantized CNN	0.43 ms per frame	2.0 W
Rangsikunpum et al. [11]	Zedboard	BNN plus GAN	0.169 ms	2.09 W
Lee et al. [12]	FPGA and 180 nm ASIC	Random forest	1.86 microseconds	Not reported
Jang et al. [13]	28 nm ASIC	Lightweight CNN	5.68 microseconds after data field	34.68 nJ per inference
Abbott McCune and Shay [8]	Altera FPGA plus Arduino Uno	Identifier ownership rule	Not reported	Not reported

builds that read the CAN bus at the bit level through a direct transceiver tap. One build runs entirely on an FPGA, one runs entirely on a microcontroller, and one splits the work between the two, with the FPGA handling the time critical tap and decision and the microcontroller handling logging, thresholds, and the bus off policy. Measuring detection latency, missed deadlines, accuracy, resource use, and power across all three builds on the same bus and the same replayed attack traffic gives the direct, controlled comparison that the reviewed literature does not provide.

Three open source projects shaped the toolchain for this three way comparison. The IDS ML repository supplies tree based training code for vehicle intrusion detection [16]. Its notebooks load the CICIDS2017 dataset, so this project wrote its own loader and training script for the Car Hacking Dataset. The FPGA random forest project generates Verilog from a trained scikit learn forest [17], but it averages one class’s leaf values instead of voting on a class label, and it no longer runs on current scikit learn releases. This project therefore wrote its own generator, which produces a pipelined voting classifier that accepts one frame per clock. The emlearn library converts the same forest into portable C for the microcontroller [18], after a small adjustment to the split thresholds so that its comparisons match scikit learn exactly. Following Araujo Filho et al., the model reads only the identifier, the data length, and the first five payload bytes, which keeps the tim-

ing budget at the 86 microseconds derived in Section 3.3 [5]. A shared set of test frames confirms that the FPGA logic, the C code, and the Python model return the same class for every frame, so all three builds run exactly one model.

References

- [1] K. Ileri, A. Rakib, and S. Djahel, MetaCAN, an optimized adaptive hybrid metaheuristic based intrusion detection system for CAN bus security, *Vehicular Communications*, vol. 55, article 100956, 2025.
- [2] C. Miller and C. Valasek, Remote exploitation of an unaltered passenger vehicle, Black Hat USA, 2015.
- [3] E. Seo, H. M. Song, and H. K. Kim, GIDS, GAN based intrusion detection system for in vehicle network, in *Proceedings of the 16th Annual Conference on Privacy, Security and Trust*, 2018, pages 1 to 6.
- [4] D. Donadel, K. Balasubramanian, A. Brighente, B. Ramasubramanian, M. Conti, and R. Poovendran, CANTXSec, a deterministic intrusion detection and prevention system for CAN bus monitoring ECU activations, *arXiv preprint arXiv 2505.09384*, 2025.
- [5] P. F. de Araujo Filho, A. J. Pinheiro, G. Kaddoum, D. R. Campelo, and F. L. Soares, An efficient intrusion prevention system for CAN, hindering cyber attacks with a low cost platform, *IEEE Access*, vol. 9, pages 166855 to 166869, 2021.
- [6] H. M. Song, H. R. Kim, and H. K. Kim, Intrusion detection system based on the analysis of time intervals of CAN messages for in vehicle network, in *Proceedings of the International Conference on Information Networking*, 2016, pages 63 to 68.
- [7] D. Lee, C. Han, and S. Lee, Hardware design of intrusion detection system for automotive CAN bus using random forest, in *Proceedings of the International Conference on Electronics, Information, and Communication*, 2023.
- [8] S. Abbott McCune and L. A. Shay, Intrusion prevention system of automotive network CAN bus, in *Proceedings of the IEEE International Carnahan Conference on Security Technology*, 2016.
- [9] M. D. Pese, B. Gozubuyuk, E. Andrechek, H. Olufowobi, M. Hamad, and K. G. Shin, MichiCAN, spoofing and denial of service protection using integrated CAN controllers, in *Proceedings of the 55th Annual IEEE and IFIP International Conference on Dependable Systems and Networks*, 2025, pages 443 to 456.
- [10] S. Khandelwal and S. Shreejith, A lightweight multi attack CAN intrusion detection system on hybrid FPGAs, in *Proceedings of the International Conference on Field Programmable Logic and Applications*, 2022, pages 425 to 429.
- [11] A. Rangasikunpum, S. Amiri, and L. Ost, BIDS, an efficient intrusion detection system for in vehicle networks using a two stage binarized neural network on low cost FPGA, *Journal of Systems Architecture*, vol. 156, article 103285, 2024.
- [12] J. Lee, S. Park, S. Shin, H. Im, J. Lee, and S. Lee, ASIC design for real time CAN bus intrusion detection and prevention system using random forest, *IEEE Access*, vol. 13, pages 129856 to 129869, 2025.
- [13] Y. Jang, H. Im, J. Kim, S. Kim, E. Kim, and S. Lee, Hardware software co optimized lightweight real time CAN intrusion detection and prevention system for ECUs, *Electronics*, vol. 15, no. 10, article 2108, 2026.
- [14] R. Kurachi, Y. Matsubara, H. Takada, N. Adachi, Y. Miyashita, and S. Horiata, CaCAN, centralized authentication system in CAN, in *Proceedings of the 14th International Conference on Embedded Security in Cars*, 2014, pages 1 to 9.
- [15] W. L. Lambert, S. Ghafoor, and S. Hollifield, Bitrate hopping, an intrusion prevention technique to secure the CAN bus based distributed network, in *Proceedings of the 24th International Symposium on Parallel and Distributed Computing*, 2025.
- [16] Western OC2 Lab, IDS ML, intrusion detection system development using machine learning algorithms, GitHub repository, github.com/Western-OC2-Lab/Intrusion-Detection-System-Using-Machine-Learning.
- [17] J. B. Snyder, FPGA random forest, an FPGA implementation of an SKLearn random forest that generates Verilog directly from a trained model, GitHub repository, github.com/johnbensnyder/FPGA_random_forest.
- [18] J. Nordby, emlearn, a library for converting trained scikit learn models, including random forests, into portable C code for microcontrollers, GitHub repository, github.com/emlearn/emlearn.